

HYDRA: A Heterogeneous Chiplet DSE Framework for Serving Dynamic Hybrid LLM Workloads

Abstract—Hybrid Transformer–Mamba large language models (LLMs) enhance long-context efficiency, but their heterogeneous computation and communication patterns complicate efficient hardware acceleration. Chiplet-based architectures offer a scalable solution by integrating specialized compute and memory units. However, the design space spanning static architectural configurations and dynamic runtime policies is prohibitively large to explore exhaustively. To address this challenge, we present HYDRA, a comprehensive design space exploration framework for hybrid LLM serving on heterogeneous chiplet systems. HYDRA jointly explores chiplet composition, placement, inter-chiplet bandwidth provisioning, dynamic batching, and runtime scheduling. It integrates communication-aware placement, dynamic batching, elastic task scheduling, and a fast Markov-based performance estimator that captures multi-tenant runtime dynamics for efficient and accurate exploration. HYDRA achieves up to $2.3\times$ higher throughput and 40% lower time-to-first-token compared to state-of-the-art baselines. These results highlight that co-designing architecture and runtime policies is critical for efficient large-scale LLM serving on heterogeneous chiplet systems.

I. INTRODUCTION

The rapid growth of large language models (LLMs) is exposing fundamental limitations in existing hardware systems for efficient high-throughput, low-latency serving. Transformer-based models remain the dominant foundation and are widely deployed on graphics processing units (GPUs), tensor processing units (TPUs), and accelerators tailored for autoregressive workloads. State-space models (SSMs), including S4 [1] and Mamba [2], have recently emerged as compelling alternatives by offering linear computational complexity and competitive performance for long-context inference. Unlike self-attention, whose KV-cache storage and memory traffic grow with context length, SSM layers maintain a compact recurrent state and provide linear-time sequence processing. As a result, SSMs substantially reduce memory pressure during long-context inference, making them increasingly attractive for high-throughput LLM serving. Their sequential dataflow and reduced memory footprint have motivated specialized accelerator micro-architectures [3]–[5]. Building on these advances, recent LLMs increasingly adopt hybrid Transformer–Mamba architectures (e.g., Jamba [6], Nemotron-H [7], and Zamba [8]). These new models introduce heterogeneous execution patterns within a single model by combining the scalable long-context processing of SSMs with the strong dependency modeling of transformers.

Accelerating hybrid LLMs is challenging due to the heterogeneity across their model components and runtime serving phases. On the model side, SSM blocks exhibit a growing proportion of element-wise operations as sequence length grows [3]. In contrast, Transformer computations are dominated

by matrix multiplications [9]. Mamba further introduces non-linear operators, such as gating and activation functions, which increase control complexity and hardware area. The prefill and decode phases further amplify this heterogeneity [10]. The prefill stage, especially for long contexts, is compute-intensive for both Transformer and Mamba blocks due to the high degree of parallel token processing. During decoding, however, the two model components exhibit fundamentally different behaviors. Mamba blocks sustain nearly constant compute and memory demand regardless of context length, whereas Transformer blocks incur linearly increasing memory and bandwidth overhead due to KV-cache access. These model- and phase-dependent behaviors necessitate the *co-design of heterogeneous chiplet architectures and runtime scheduling policies to effectively exploit workload characteristics*.

The integration of diverse processing units onto a single monolithic chip for large-scale LLM workloads has become increasingly cost-prohibitive [11]. Consequently, heterogeneous chiplet-based architectures have emerged as a promising solution for accelerating multi-tenant hybrid LLM services by enabling modular integration of compute, memory, and networking components [12]. However, designing chiplet-based systems for hybrid LLM serving requires the joint co-design of architecture and runtime scheduling, significantly expanding the design space. Specifically, *macro-architecture* design space exploration (DSE) for large-scale heterogeneous platforms introduces two key challenges:

(I) Broad Design Space: Heterogeneous architectures for hybrid LLM workloads require heterogeneous chiplets optimized for Transformer and Mamba computations, each further tailored to prefill and decode phases. In addition, the area allocated to compute chiplets must be balanced against memory capacity and inter-chiplet communication resources. The number of candidate configurations grows rapidly with both system scale and architectural heterogeneity. Fig. 1(a) shows the maximum number of configurations that can be explored under a given simulation time budget. Even with fixed chiplet placement and scheduling policies, exhaustive simulation of large-scale heterogeneous systems can require days to weeks of wall-clock time on a 64-core CPU [13]. Therefore, naïve DSE becomes impractical, motivating the need to reduce exploration time from weeks to minutes.

(II) Runtime Unpredictability: Auto-regressive decoding depends on input-dependent behavior, while multi-tenant serving introduces asynchronous request arrivals with diverse prefill and decode characteristics [14]. These factors cause highly dynamic resource demands across computation, memory, and inter-chiplet communication, limiting the effectiveness of of-

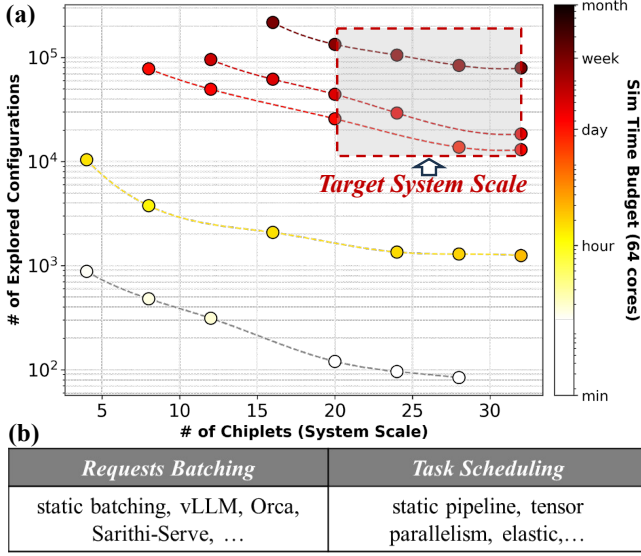


Fig. 1: (a) Design space of the chiplet-based system's macroarchitecture and corresponding simulation cost (b) Runtime and design-time scheduling strategies for LLM services.

fine task mapping and static performance models commonly used in DSE frameworks. As illustrated in Fig. 1(b), modern LLM serving relies on dynamic batching (e.g., vLLM [15], Orca [16], Sarithi-Serve [17]) and diverse task scheduling strategies, necessitating runtime-aware modeling for accurate evaluation. *Existing DSE frameworks for chiplet-based systems largely assume static workloads and fixed scheduling policies*, and therefore fail to capture the dynamic, multi-tenant behavior of hybrid LLM serving [12], [18], [19].

To address these challenges, we propose *HYDRA*, a DSE framework for heterogeneous chiplet-based systems targeting hybrid LLM serving. *HYDRA* integrates communication-aware chiplet placement, dynamic request batching, and elastic task scheduling with a high-level simulator and a Markov-based performance estimator that guide the efficient and accurate DSE. The Markov-based estimator is not intended to replace detailed simulation. Instead, it serves as a fast pruning and ranking mechanism that rapidly identifies promising regions of the design space. We complement it with detailed simulations to validate selected configurations and accurately capture transient contention, NoI communication effects, memory-management behavior, dynamic batching decisions, and runtime pipeline imbalance that are intentionally abstracted by the estimator. Together, these components capture runtime variability and enable efficient co-exploration of architecture and runtime policies. *HYDRA* identifies Pareto-optimal configurations in the throughput and time-to-first-token (TTFT) space. Across all evaluated workloads, *HYDRA* achieves up to $2.3\times$ higher throughput and 40% lower TTFT compared to baselines. To accelerate exploration, we develop a Markov-based performance estimator that characterizes runtime transitions among chiplet states and predicts system throughput. This estimator achieves an average cosine similarity of 0.9 with full simulation results and reduces DSE time from days to minutes.

Main contributions of this work are:

- A novel open-source DSE framework for hybrid LLM workloads on 2.5D heterogeneous chiplet systems,
- A communication-aware chiplet placement strategy, combined with dynamic batching and elastic task scheduling, to improve compute and memory utilization,
- A Markov-based performance estimator for hybrid LLM workloads that enables fast and accurate DSE,
- Comprehensive evaluations across hybrid, Mamba, and Transformer models on diverse datasets.

II. RELATED WORK

Hybrid Transformer-Mamba LLMs leverage Transformer-style attention for in-context learning with Mamba layers that enable recurrent linear-complexity sequence modeling. This fusion improves scalability to long context lengths while reducing memory overhead compared to pure Transformer baselines. Models with a larger Attention fraction tend to favor higher memory capacity and bandwidth due to KV-cache traffic, whereas Mamba-dominated models place greater emphasis on recurrent-state processing and sustained decoding throughput. For example, Jamba integrates the hybrid architecture with Mixture-of-Experts (MoE) layers, achieving competitive accuracy with up to $3\times$ higher throughput [6]. Nemotron-H and Minitron-SSM employ a similar hybrid strategy, introducing a unified SSM pruning pipeline that compresses the model by 50% with minimal quality impact [7], [20].

Significant research has focused on optimizing the prefill and decode phases of Transformer and Mamba accelerators. Dedicated accelerators have been proposed for either transformer (e.g., TSTC [21], and AccelTran [22]) or Mamba (e.g., LightMamba [4], SSM-RDU [23], eMamba [5], and SpecMamba [24]). Recent approaches propose unified architectures that efficiently support both SSM computation and matrix multiplication. For example, MARCA introduces a reconfigurable tensorcore-like architecture that accelerates element-wise and specific nonlinear operations [3]. Geens et al. further explore MARCA's design space and reveal how different allocations of compute and on-chip memory resources shift bottlenecks across serving phases [25]. While these works focus on accelerator micro-architecture, *HYDRA focuses on system-level macro-architecture DSE for hybrid LLM serving*. Unlike prior work, our design space explicitly captures heterogeneous accelerator configurations across both prefill and decode phases. These observations highlight the need for system-level frameworks that jointly optimize how heterogeneous compute and memory resources are composed, placed, and scheduled in chiplet-based systems for end-to-end LLM serving.

Determining the optimized composition of heterogeneous chiplets for hybrid LLM workloads remains an open problem. Prior DSE frameworks have explored different parts of the design space of chiplet-based systems. Gemini adopts a scalable Simba-like architecture and performs layer-wise pipeline mapping for DNN workloads, targeting homogeneous compute chiplets [18]. Focusing on edge deployments, Cascade clusters multi-tenant workloads based on their compute and memory

TABLE I: Comparison of prior chiplet-based DSE frameworks across workload support, heterogeneity, scalability, target design space, and runtime-dynamics coverage.

Framework	Workload	Hetero.	Scale	Target DS	Runtime Dyn.
Gemini	CNN/MLP	×	8 chiplets	Microarch.	×
Cascade	Edge App.	✓	12 chiplets	Macroarch.	×
WSC-LLM	Transformer LLM	×	64 chiplets	Microarch.	×
HYDRA	Hybrid LLM	✓	24 chiplets	Macroarch.	✓

characteristics, and assigns them across chiplet packages [19]. WSC-LLM targets LLM workloads deployed on wafer-scale chips and explores micro-architectural trade-offs across DRAM capacity, communication bandwidth, and compute resources distributed across chiplets [12]. It also proposes a disaggregated scheduling strategy that separates prefill and decode execution, an approach also used in modern GPU serving systems. *Despite these advances, existing DSE frameworks largely assume static workload traces and do not capture the dynamic, multi-tenant behavior of LLM serving* (e.g., varying prefill-decode lengths and asynchronous request arrivals). As summarized in Table I, *HYDRA* addresses both runtime dynamics and macro-architecture exploration, which are essential for efficient hybrid LLM serving on heterogeneous chiplet systems.

Recent LLM evaluation and scheduling frameworks also aim to capture LLM-specific dynamics in system behavior. For example, Vidur is a simulation framework for LLM inference on GPUs that integrates a workload generator with various runtime schedulers to model realistic multi-request serving scenarios [26]. Our work complements these efforts by coupling hybrid LLM workload modeling with chiplet-aware DSE and runtime scheduling co-design, enabling a holistic analysis of performance bottlenecks and system-level trade-offs in heterogeneous chiplet architectures.

III. HYDRA DSE FRAMEWORK

A. Overview

The *HYDRA* framework takes two classes of inputs, as illustrated in Fig. 2: (1) the workload specification, including the target models, datasets, and request traces, (2) system constraints, including the chiplet library, network-on-interposer (NoI) topology, interposer size, and die-area budget. Given these inputs, *HYDRA* performs joint exploration across four architectural and runtime dimensions:

- 1) chiplet placement (Section IV-A),
- 2) request batching policy (Section IV-B),
- 3) task scheduling strategy (Section IV-C),
- 4) the macro-architecture, including chiplet types and quantities (Section III-D).

Each candidate configuration is evaluated using a combined simulation and performance estimation flow that captures prefill/decode behaviors, multi-tenant concurrency, and inter-chiplet communication. Feasible configurations are compared in terms of throughput and TTFT. This modular structure enables *HYDRA* to capture the interactions between architecture, runtime scheduling, and workload dynamics during exploration.

Overall, *HYDRA* models hybrid LLM serving as a tightly coupled system of (i) workload dynamics, (ii) heterogeneous

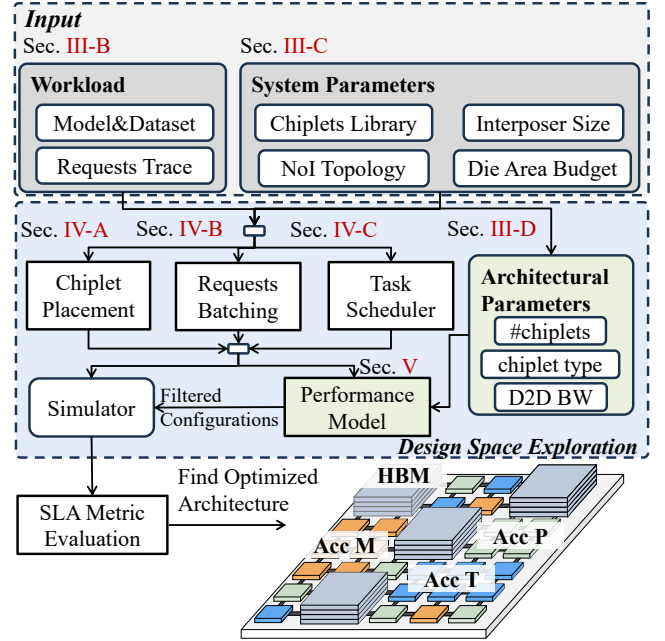


Fig. 2: The overview of the proposed design space exploration framework targeting hybrid LLM services.

chiplet resources, and (iii) runtime batching/scheduling policies. The DSE process jointly explores architectural configurations and runtime decisions under system constraints to optimize system-level throughput and TTFT.

B. Workload Description

We model hybrid LLMs serving workloads with diverse computational and memory behaviors across application domains. The proposed workload model captures both structural heterogeneity (across model components) and temporal dynamics (across requests and serving phases), as described next.

Models: *HYDRA* takes a layer- or block-level description, where each block is annotated with its compute, memory, and communication requirements. This representation supports heterogeneous LLMs composed of different operator types and execution patterns, including Attention-based blocks (*A*), Mamba-based blocks (*M*), or their combinations. Example models used for evaluation are described in Section VI-A.

Datasets: We employ commonly used benchmarks that cover diverse inference patterns, including summarization, translation, interactive chat, and long-form generation, each with distinct prefill/decode characteristics. *HYDRA* also supports a configurable request arrival process to control the relative intensity of prefill and decode.

Service dynamics: Each request undergoes a *prefill phase*, where the full input sequence is processed once, followed by a *decode phase*, where tokens are generated autoregressively. In multi-tenant scenarios, decode-phase duration and resource requirements are input-dependent, while chiplet utilization varies dynamically as requests arrive and complete. These temporal variations drive the compute, memory, and communication demands, making runtime scheduling and batching essential for efficient system utilization.

Task Definition: *HYDRA* translates workloads into task-level execution streams used for both simulation and performance estimation. Each task k represents all compute and memory operations of a model operation during inference, such as a fully connected layer, an Attention, or SSM operation. $\mathcal{K}$ denotes the set of all schedulable tasks generated during inference for the target model.

C. System Specification

The system configuration is specified by four aspects: the chiplet library, the NoI, the interposer size, and the die area budget. The chiplet library and NoI determine the system's architectural capabilities and heterogeneity, while the interposer size and die-area budget impose physical and cost constraints on feasible designs.

Chiplet Library: *HYDRA* maintains a library of compute and memory chiplets. Compute chiplets may differ in accelerator organization, phase specializations (e.g., optimized for prefill or decode), and target model component (e.g., optimized for Transformer or SSM blocks). This abstraction enables *HYDRA* to model a broad range of candidates, including commercial-inspired designs (e.g., B200-like [27]), prior accelerator architectures (e.g., TSTC [21] and MARCA [3]), and customized systolic-array- or vector-based architectures. Memory chiplets may vary in technology, capacity, and bandwidth, such as High Bandwidth Memory (HBM3 [28]) and GDDR7 [29].

NoI Specification: *HYDRA* models the NoI, including topology (e.g., mesh and torus [30]), die-to-die (D2D) bandwidth, latency, and communication cost. These parameters determine the efficiency of data movement across chiplets, including weights, activations, KV cache, and recurrent states.

D2D bandwidth provisioning also affects chiplet microarchitecture under a fixed die-area budget. Increasing bandwidth requires additional communication hardware, such as PHYs, micro-bumps, and routing logic, which reduces the area available for compute units and on-chip SRAM. Consequently, bandwidth, computation, and local buffering must be co-optimized during DSE. For illustration, Fig. 3 shows how D2D bandwidth provisioning affects the area breakdown of MARCA [3] chiplets optimized for prefill and decode. *These trade-offs directly influence the optimal chiplet composition and placement decisions explored during DSE.*

Interposer Size: *HYDRA* uses the interposer size to constrain the physical integration space of the chiplet system. This parameter bounds the number of chiplets that can be integrated and affects their feasible placement, shaping the architectural design space explored by the framework. The concrete interposer assumptions used in evaluation are given in Section VI-A.

Die Area Budget: Each chiplet in *HYDRA* is subject to a die-area budget that limits the amount of compute, on-chip memory, and communication hardware that can be integrated. Together with the NoI specification, this budget determines the trade-off among compute resources, SRAM capacity, and communication interfaces within each chiplet.

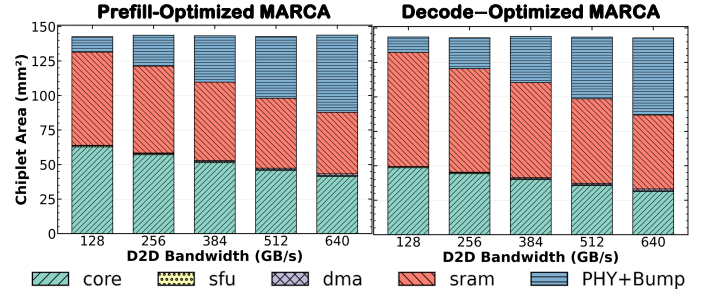


Fig. 3: The area breakdown of compute chiplets in different NoI bandwidth. Cores and SFUs are compute units for linear and non-linear operations. SRAM represents local buffers. DMA, PHY, and bumps represent communication components.

D. Design Space

This section defines the macro-architecture design space explored by *HYDRA*. It captures how compute, memory, and communication resources are provisioned at the package level. An architectural configuration is defined by three key aspects:

- **Chiplet types:** The set of available chiplet types, including prefill-optimized Attention (A_p), decode-optimized Attention (A_d), prefill-optimized Mamba (M_p), and decode-optimized Mamba (M_d) accelerators, as summarized in Table II.
- **Chiplet composition:** The number of instantiated chiplets for each type, which determines the overall mix of compute and memory resources in the system. For example, a composition defines the instantiated accelerator sets C_A and C_M for Attention and Mamba, respectively.
- **NoI bandwidth:** The D2D bandwidth provisioning under a given topology (e.g., 128 or 256 GB/s links).

These architectural choices are constrained by the interposer area budget, i.e., the total area of all instantiated chiplets must not exceed the available interposer area. Under a given topology and area constraint, each architectural configuration is defined by the selected chiplet types, their composition, and D2D bandwidth. For each such configuration, *HYDRA* further explores runtime and physical design decisions, including chiplet placement, dynamic request batching, and elastic task scheduling (Section IV).

TABLE II: Summary of notations used in this work

Notation	Definition
$k \in \mathcal{K}$	Schedulable task instance (k) and set of tasks ($\mathcal{K}$).
A_p, A_d	Prefill-/decode-optimized (A_p/A_d) Attention accelerators
M_p, M_d	Prefill-/decode-optimized (M_p/M_d) Mamba accelerators
C_A	The set of Attention accelerator chiplets
C_M	The set of Mamba accelerator chiplets
Loc_M, Loc_A	The set of locations of HBM chiplets allocated to the Mamba blocks (Loc_M) or Attention blocks (Loc_A)
r_M, r_A	The maximum request concurrency supported by the Mamba HBM group (r_M) or Attention HBM group (r_A)
$v(k, c)$	Communication volume of task k on chiplet c .
$p(c)$	Chiplet placement location of c
$t_{comp}(k, c)$	Compute latency of task k on chiplet $c \in C_A \cup C_M$
$t_{comm}(k, c)$	Communication latency of task k on chiplet c
$\Omega(k)$	Task assignment of k to a chiplet

Overall, *HYDRA* jointly explores architectural configurations and runtime policies to capture their interactions and identify designs that optimize throughput and TTFT. Each candidate configuration can be evaluated via full simulation, which models dynamic request arrivals, batching behavior, and elastic scheduling. However, as shown in Fig. 1, simulation becomes prohibitively time-consuming for large-scale systems and dynamic LLM workloads. Therefore, we use a fast and accurate performance model, introduced in Section V.

IV. LLM INFERENCE OPTIMIZATION STRATEGIES

A. Chiplet Placement

HYDRA models chiplet placement on a 2D grid-based interposer layout, where each grid cell represents the minimum allocatable unit. A chiplet may occupy one or multiple adjacent cells depending on its physical footprint, enabling us to capture diverse chiplet sizes and layout constraints. For example, each cell can be 5 mm x 5 mm, and the chiplet of size 100 mm² can occupy four of these cells. This abstraction supports a flexible representation of candidate placements.

Given the chiplet composition, the placement objectives are: (i) maximize the number of concurrent requests processed under memory constraints, and (ii) minimize inter-chiplet communication cost. These objectives are tightly coupled and depend on workload characteristics, making joint optimization intractable for large design spaces. To make this problem tractable, we decompose placement into two stages that separately address memory capacity constraints and communication locality, as illustrated in Fig. 4.

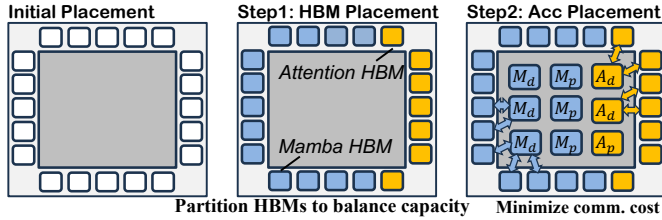


Fig. 4: The overview of the chiplet placement in *HYDRA*

Step 1: HBM Placement. HBM chiplets are placed along the outer ring of the interposer to reflect their role as high-bandwidth memory endpoints with efficient access to package-level interfaces and power delivery. This organization is common in prior chiplet-based systems [27], [31]. Given these locations, we partition the HBM chiplets into Mamba and Attention groups to provision memory for their respective workloads. Since these workloads exhibit distinct memory-access patterns and runtime bottlenecks, separating memory resources reduces interference and improves utilization.

For a candidate partition (N_M, N_A) , where N_M and N_A denote the number of HBM chiplets assigned to Mamba and Attention, respectively, we estimate the maximum concurrent requests supported by each group. We first estimate the maximum concurrency supported by the Mamba HBM group, denoted by r_M , as:

$$r_M = \max \left(0, \left\lfloor \frac{N_M \cdot C_{HBM} - L_M \cdot W_M}{S_M} \right\rfloor \right) \quad (1)$$

where C_{HBM} is the capacity of a single HBM, L_M is the number of Mamba blocks in the model, W_M is the weight memory required by one Mamba block, and S_M is the per-request memory required to store the Mamba recurrent states. Here, $N_M \cdot C_{HBM}$ denotes the total HBM capacity assigned to Mamba, while $L_M \cdot W_M$ denotes the memory reserved for all Mamba weights. Therefore, the numerator is the remaining HBM capacity available for runtime state allocation, and r_M gives the maximum concurrency supported by the Mamba HBM group.

The maximum concurrency for the Attention HBM group, denoted by r_A , is estimated in the same manner by replacing the Mamba weight and recurrent-state terms with the corresponding weights and KV-cache terms. Then, we select the partition (N_M, N_A) that maximizes $\min(r_M, r_A)$ to avoid memory bottlenecks in either component. Once the best partition is found, the corresponding outer-ring HBM locations are labeled as the Mamba HBM group and the Attention HBM group, and are denoted Loc_M and Loc_A , respectively. These two sets of locations are then passed to Step 2.

Step 2: Communication-Aware Placement. Given the HBM locations from Step 1, compute chiplets are placed to minimize the inter-chiplet communication overhead. The objective is to place each compute chiplet close to the memory resources it accesses most frequently.

Let C_M and C_A denote the sets of Mamba and Attention compute chiplets (defined in Table II). Thus, $C_M \cup C_A$ constitutes the full set of compute chiplets to be placed. Consider the placement of Mamba compute chiplets C_M for illustration. We first select candidate locations for all Mamba compute chiplets based on the smallest Manhattan distance to Loc_M . For a given task $k \in \mathcal{K}$, the objective is to find the placement that minimizes the total weighted communication cost:

$$\arg \min_p \sum_{c \in C_M} \sum_{h \in Loc_M} d(p(c), h) \cdot v(k, c) \quad (2)$$

where $p(c)$ denotes the location of chiplet c , $d(\cdot, \cdot)$ is the Manhattan distance between two locations, and $v(k, c)$ is the communication volume required when task k is executed on chiplet c . For example, k denotes the state-space model [1] computation when placing the Mamba chiplets C_M .

The placement of the Attention chiplets in C_A is determined in the same manner using Loc_A . Because C_M and C_A are placed at the same grid-based interposer layout, their preferred locations may overlap. To resolve location conflicts, we place C_M and C_A sequentially in descending order of their sizes, allowing larger groups to claim preferred locations first. After one group is placed, its assigned locations are removed from the available set before constructing the candidate location set for the other group. Overall, this strategy co-locates compute chiplets with their dominant communication partners, reducing NoI traffic and latency. Section VI-D evaluates the effectiveness of this placement strategy.

In summary, the proposed two-stage decomposition separates memory capacity balancing from communication optimization, enabling efficient exploration while capturing the dominant performance bottlenecks in hybrid LLM serving.

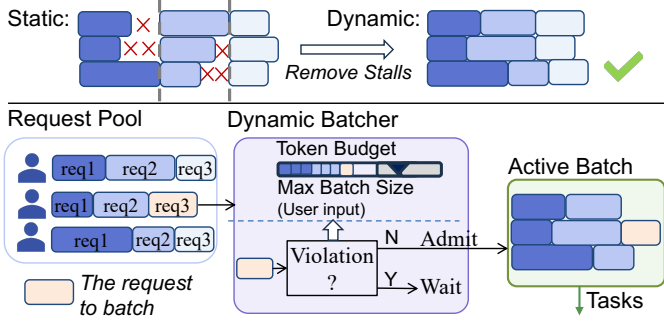


Fig. 5: The overview of the *dynamic batching* in *HYDRA*

B. Dynamic Request Batching

Request batching is a key optimization in LLM serving because it improves parameter reuse and exposes token-level parallelism. However, in multi-tenant scenarios, requests arrive asynchronously and exhibit highly variable execution costs: input lengths differ, decode lengths are input-dependent, and resource demand cannot be known a priori. Under these conditions, static batching is inefficient, as illustrated in the top part of Fig. 5. Static execution is often gated by the longest request, while shorter requests complete early but cannot be replaced. This leads to stalls and underutilization of compute, memory, and communication resources.

To address this limitation, *HYDRA* adopts dynamic request batching, maintaining a continuously evolving active batch rather than fixed batch boundaries. As shown in Fig. 5, requests are admitted as soon as resources become available, without waiting for batch-level synchronization. Incoming requests first enter a request pool with varying input sizes and memory requirements. A request is admitted into the active batch if two conditions are met:

- 1) The active batch size is smaller than a user-defined maximum since larger batches can significantly degrade per-request prefill latency, as evaluated in Section VI-B,
- 2) Adding the new request does not violate the available token budget.

We compute a token budget as the product of the current batch size and median per-request token cost. New requests are admitted only when the estimated token demand remains below a system-level token budget derived from available compute, memory, and communication resources.

To enable continuous admission without the overhead of frequent reallocation, memory management is decoupled from request admission via virtualized memory blocks, similar to vLLM [15]. Each request is assigned virtual memory for its execution state, which grows dynamically with KV-cache or recurrent state during decoding, while physical memory is allocated on demand. This design avoids large contiguous allocations and improves robustness to variable request footprints.

HYDRA employs request-level preemption to prevent out-of-memory conditions. If admitting a new request or expanding an existing request’s KV/state storage would exceed the available HBM capacity, *HYDRA* preempts the request with the fewest generated tokens, reclaims its memory allocation, and returns

it to the request pool. This policy prioritizes requests that are closer to completion, reducing wasted work while maintaining forward progress under memory pressure. Whenever requests are admitted, completed, or preempted, the active batch is updated accordingly. The resulting batch snapshot is then passed to the elastic task scheduler for execution.

Overall, *HYDRA*’s dynamic batching policy generalizes continuous batching to heterogeneous chiplet systems and phase-dependent hybrid LLM workloads, explicitly incorporating trade-offs among compute, memory, communication, and latency into admission control.

C. Elastic Task Scheduler

After request admission, *HYDRA* dispatches schedulable tasks $k \in \mathcal{K}$ to heterogeneous compute chiplets. We characterize the compute latency $t_{\text{comp}}(k, c)$ and communication latency $t_{\text{comm}}(k, c)$ for each task–chiplet pair (k, c) . Task k can denote a phase-specific model-block operation, such as any Attention or SSM computation. We obtain $t_{\text{comp}}(k, c)$ from our runtime measurements or estimate from published specifications for commercial chiplets [27], and published microarchitectural models for prior accelerators [3], [21]. For our custom accelerator designs, we implement the datapaths in RTL and obtain cycle-level latency from RTL-level simulation. These values form a task–chiplet latency table that captures the affinity between each task type and compatible chiplet. The profiled latency also includes chiplet-local memory access under the configured external bandwidth. We estimate $t_{\text{comm}}(k, c)$ using the following equation:

$$t_{\text{comm}}(k, c) = d_{\text{Man}}(c, m_k) \frac{V_k}{B_{\text{NoI}}}$$

where V_k is the data volume transferred by the task, $d_{\text{Man}}(c, m_k)$ is the Manhattan distance to its memory chiplet, and B_{NoI} is the provisioned per-link NoI bandwidth. Thus, the Manhattan distance weights the transfer cost by the number of links along the path. For tasks accessing multiple memory objects or memory chiplets, these costs are accumulated. *HYDRA* uses $t_{\text{comp}}(k, c) + t_{\text{comm}}(k, c)$ to construct the initial mapping. Runtime contention is subsequently captured by the simulator.

The first objective is to find an initial mapping for each ready task using the performance profiles, as illustrated in Fig. 6. For a given mapping, the cumulative latency assigned to chiplet c can be estimated as:

$$t_c^\Omega = \sum_{k: \Omega(k)=c} (t_{\text{comp}}(k, c) + t_{\text{comm}}(k, c)) \quad (3)$$

We aim to minimize the bottleneck stage across chiplets:

$$\arg \min_{\Omega} \max_{c \in C_M \cup C_A} t_c^\Omega \quad (4)$$

Therefore, *HYDRA* solves this optimization problem to construct an initial spatial pipeline mapping $\Omega(k)$ that stores the preferred chiplet assignment for each task [18], [19]. This mapping would be optimal for a static system where the chiplet utilization and ready tasks are known a priori.

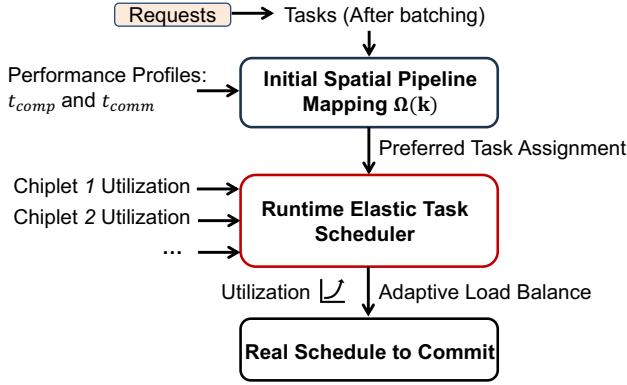


Fig. 6: The overview of the task scheduling in *HYDRA*

However, *HYDRA* targets multi-tenant hybrid LLM serving, where the runtime mix of prefilling and decoding requests can vary significantly over time. Under such dynamic conditions, the preferred task assignment may create load imbalance and reduce overall chiplet utilization. Therefore, *HYDRA* uses a runtime elastic task scheduler to modify the preferred assignments, as illustrated in Fig. 6. The elastic scheduler monitors chiplet utilization through the runtime queue status of compute chiplets. When load imbalance is detected, it adaptively balances the preferred assignments by scheduling more tasks to underutilized chiplets.

In summary, the *initial* spatial pipeline mapping provides a *preferred task assignment before execution*, while the *runtime elastic task scheduler corrects transient imbalance* caused by dynamic request arrivals and varying prefill/decode pressure. The resulting execution schedule can therefore adapt to runtime conditions without requiring expensive global remapping. This improves chiplet utilization and, in turn, improves throughput and TTFT, as demonstrated in Section VI-E.

V. PERFORMANCE ESTIMATION FOR FAST DSE

Evaluating each candidate configuration with full event-driven simulation is prohibitively expensive for large-scale chiplet systems and dynamic LLM workloads. Full simulations must capture asynchronous request arrivals, dynamic batching, elastic scheduling, memory behavior, and NoI contention, leading to hours of exploration time per design point. In contrast, static analytical models (e.g., roofline approach [32]) are fast but fail to capture runtime dynamics, leading to inaccurate performance estimates. Existing approaches also do not capture the coupling between batching, scheduling, and resource contention, which is critical in multi-tenant LLM serving [18], [19]. So, there is a strong need for techniques that explicitly incorporate dynamic batching, elastic scheduling, and heterogeneous chiplet capabilities in a unified formulation.

To address this challenge, we develop a lightweight *Markov-based performance estimator* that captures the dominant runtime dynamics while enabling fast DSE, as illustrated in Fig. 7. The key abstraction is reducing the high-dimensional scheduling problem into a low-dimensional state space defined by chiplet allocation. It is based on the observation that, hybrid LLM serving performance is primarily governed by how

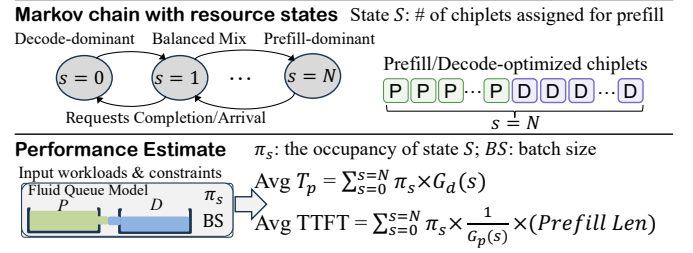


Fig. 7: Overview of the Markov-based performance estimation.

resources are split between prefilling and decoding rather than by the exact ordering of individual tasks. Dynamic batching changes the amount of active work in each phase, while elastic scheduling changes which chiplets serve that work. Therefore, the dominant effect of runtime control can be captured by tracking aggregate chiplet allocation across execution phases.

A. Modeling Elastic Chiplet Allocation

Under elastic scheduling, system performance is governed by how compute chiplets are dynamically partitioned between prefilling and decoding. We model this behavior as a continuous-time Markov chain (CTMC) [33], where each state represents a particular allocation of chiplets across execution phases. This formulation captures the dominant system dynamics by modeling chiplet allocation as the primary state variable.

Let N be the total number of compute chiplets. A system state is defined by $s \in \{0, \dots, N\}$, representing the number of chiplets assigned to prefill, while the remaining $N - s$ chiplets serve decode. With elastic scheduling, chiplets can be reassigned across phases when demand exceeds the preferred allocation. When prefilling demand exceeds available prefill-oriented chiplets, decoding chiplets are temporarily reassigned, and vice versa. This allows the system to adapt to dynamic workloads. This behavior directly reflects the elastic scheduling and dynamic batching mechanisms described in Section IV.

B. State-Dependent Service Capacity

For each state s , we estimate the system's aggregate processing capacity based on the chiplet types and their assigned roles. Recall that there are prefill- and decode-optimized chiplets. Let $c_{p,P}$ and $g_{p,P}$ denote the number active *prefill-optimized chiplets* assigned to *prefill tasks* and their *prefill token processing rates*. Similarly, let $c_{d,D}$ and $g_{d,D}$ denote the number active *decode-optimized chiplets* assigned to *decode tasks* and their *decode token processing rates*. Elastic scheduling allows *HYDRA* to use decode-optimized chiplets for prefill tasks and vice versa. We denote the number of decode-optimized chiplets assigned to prefill tasks as $c_{p,D}$ and the corresponding processing rate as $g_{p,D}$. Similarly, the number and processing rate of prefill-optimized chiplets assigned to decode tasks are denoted by $c_{d,P}$ and $g_{d,P}$. Using these definitions, the aggregate prefilling and decoding capacities can be expressed as:

$$G_p(s) = c_{p,P} \cdot g_{p,P} + c_{p,D} \cdot g_{p,D} \quad (5)$$

$$G_d(s) = c_{d,D} \cdot g_{d,D} + c_{d,P} \cdot g_{d,P} \quad (6)$$

These rates depend on both hardware characteristics and workload properties (e.g., sequence length). Thus, they capture both

compute and memory constraints, as well as communication overheads. Since steady-state token generation is limited by decoding, $G_d(s)$ denotes the system throughput in state s .

C. Throughput and TTFT Estimation

Computing the steady-state distribution of the CTMC analytically is intractable due to the complex interaction among request arrivals, dynamic batching, and elastic scheduling. Instead, *HYDRA* employs a lightweight fluid-queue model [34] to approximate system dynamics over time. The fluid model tracks the aggregate amount of outstanding prefilling and decoding work and updates the chiplet allocation accordingly. As workloads are executed, the model tracks the fraction of time spent in each state. Let π_s denote the estimated fraction of time spent in state s . This approximation captures macroscopic system behavior without requiring fine-grained event-driven simulation. Using the prefill/decode processing rates and steady-state probabilities, the expected system throughput is:

$$T_p = \sum_{s=0}^N \pi_s \cdot G_d(s) \quad (7)$$

This formulation weights each state’s decoding capacity by its occupancy, capturing how workload dynamics and chiplet allocation jointly determine throughput. TTFT is estimated from the same state trajectory, capturing the fundamental throughput–TTFT tradeoff: allocating more chiplets to decoding improves throughput, while allocating more chiplets to prefilling reduces TTFT.

D. Generalization to Hybrid Models

The proposed modeling approach naturally extends to hybrid LLMs with multiple operator types (e.g., Mamba and Attention). In these cases, each state shows how chiplets are assigned to different task classes, and the service rates depend on the characteristics of each operator. In summary, the proposed estimator captures the main runtime dynamics of hybrid LLM serving and remains computationally efficient while accurately capturing dominant runtime behavior, enabling rapid pruning of suboptimal configurations.

VI. EXPERIMENTS

A. Experimental Setup

Input models: Evaluations are performed on three representative LLM models: (1) *Nemotron-H-4B* [7], a hybrid Transformer–Mamba model that combines Attention and SSM modules; (2) *LLaMA3-7B* [35], a pure Transformer; (3) *Mamba-2.8B* [2], an SSM-based model. These models cover hybrid, Transformer-only, and SSM-only workloads.

Input datasets: We use four datasets that capture diverse inference patterns: (1) ArXiv-4K [36] (ArXiv): long scientific documents, representing long prefill/short decode; (2) Bilingual Web Books [37] (BWB): multilingual narrative text, representing long prefill/long decode; (3) LongWriter-6K [38] (LW): extended-context writing tasks, representing short prefill/long

decode; and (4) LMSYS-Chat-1M [39] (Chat): diverse chat interactions, representing short prefill/short decode. This set spans a broad range of prefill–decode ratios and serving behaviors.

NoI Specification: We experiment on a passive silicon interposer with a 2D mesh topology, a widely adopted design for large systems [40]. Die-to-die (D2D) links follow NVIDIA GRS [41], where each 8-lane, 25-Gbps PHY occupies 0.3876 mm² in 16 nm. Micro-bump area, bandwidth, and latency follow UCIe x64 advanced-package assumptions [42]. The evaluated D2D bandwidth options are specified in Table III.

Interposer/die area constraints: We assume 2700–3000 mm² interposer area, comparable to state-of-the-art packages such as NVIDIA B200 [27] and AMD MI350 [31]. This range is comparable to state-of-the-art accelerator packages and permits large-scale heterogeneous chiplet systems. Individual chiplet area is capped at approximately 121 mm², comparable to the footprint of an HBM stack, to simplify floorplanning and avoid the steep cost increase associated with large dies, following standard chiplet cost models [43].

Chiplet library: The chiplet library comprises four LLM accelerator chiplets: MARCA [3] optimized for the prefill and decode phases of Mamba (M_p and M_d), and TSTC [21] optimized for the prefill and decode phases of Transformer (A_p and A_d). Prefill-oriented chiplets allocate area approximately evenly between compute and SRAM, whereas decode-oriented chiplets allocate roughly twice as much area to memory resources as to computation to better support bandwidth-intensive decoding.

As discussed in Section III-C, higher NoI bandwidth requires additional communication resources on the chiplet, reducing the area available for compute units and on-chip SRAM. In addition, the system employs HBM3 chiplets for model parameters, KV caches, and intermediate states. To ensure fair comparison across heterogeneous designs, all components are normalized to a 22 nm CMOS node using DeepScale [44]. This choice enables comparing designs originally proposed for different process nodes using a unified cost and area model.

LLM inference evaluation: LLM inference is evaluated using an in-house event-driven simulator that models computation, memory behavior, and NoI communication at chiplet granularity. It tracks per-chiplet task queues, execution progress, and link utilization, enabling accurate modeling of contention and overlap across computation and communication. Memory chiplet performance is characterized using Ramulator [45], with capacity and bandwidth parameters derived from vendor-level specifications [28]. On-chip SRAM buffer areas are obtained from CACTI [46], while special function units (SFUs) for non-linear operators are modeled using FlexSFU [47]. The performance of MARCA and TSTC kernels is modeled using the microarchitectural parameters and kernel-level characterizations reported in prior work, accounting for operation type, data dependencies, and available parallelism.

All simulations are conducted on an AMD Ryzen Threadripper 7985WX CPU [13]. Each configuration is simulated for 100 seconds of workload time.

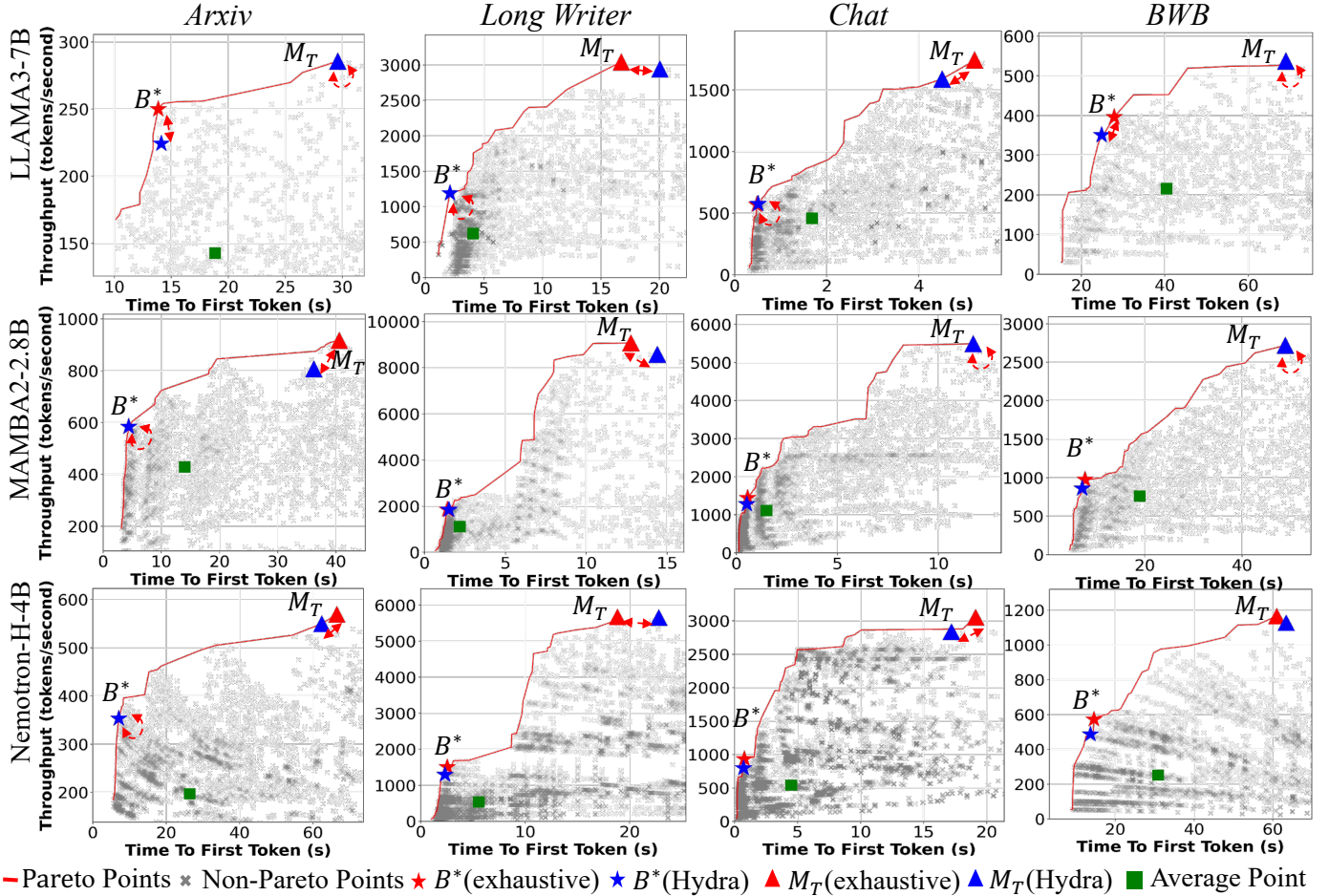


Fig. 8: Overview of the DSE on 12 workloads (model-dataset pairs). The red points are the Pareto configurations derived through the exhaustive search of macro-architectures. The blue points are the Pareto configurations estimated by the Markov-based method. B^* : Configurations achieving the highest throughput-TTFT ratio; M_T : Configurations achieving the best throughput.

TABLE III: Design space exploration parameters. Some configurations are shared across all models.

Chiplet Counts	LLAMA3	MAMBA2	Nemotron-H
Total		24	
N_{Ap} (Prefill)	1 – 21	–	1 – 19
N_{Ad} (Decode)	1 – 21	–	1 – 19
N_{Mp} (Prefill)	–	1 – 21	1 – 19
N_{Md} (Decode)	–	1 – 21	1 – 19
N_{HBM3}		2 – 16	
Other Configs			
D2D BW	256, 384, 512, 640 GB/s		
Max Batch Size	2, 4, 8, 16, 32, 64		

B. The DSE on Macro-architectures

The goal of this section is to analyze how chiplet composition, NoI bandwidth provisioning, and batch sizing shape the throughput-TTFT trade-off when dynamic optimizations are disabled. Hence, we explore macro-architectural configurations under *static request batching* and *static task scheduling* [19] to expose fundamental architectural trade-offs, using the design parameters summarized in Table III.

Methodology: To efficiently explore this design space, we first apply a fast performance estimation stage to identify candidate

Pareto-optimal configurations in the throughput (TP)-TTFT space. This stage uses our Markov-based estimator (Section V) to approximate system performance, reducing the number of simulated design points by 2520–8520 $\times$ relative to exhaustive exploration. The proposed Markov-based estimator is sufficiently general to model both dynamic and static execution. In this experiment, static batching and scheduling correspond to a special case in which chiplet allocation remains fixed and runtime reassignment is disabled. For validation, we compare the predicted Pareto points against exhaustive event-driven simulation that enumerates all feasible configurations under the same constraints.

Design trade-off: Fig. 8 summarizes the macro-architecture design space exploration results across all model-dataset pairs. Each subplot shows the throughput and TTFT of all simulated configurations. The red curve traces the Pareto frontier obtained from exhaustive simulation, while the gray points correspond to dominated configurations. Across all workloads, configurations that increase throughput typically allocate more resources to decoding, either through larger effective batch sizes or higher NoI bandwidth. While this improves steady-state token generation, it often increases TTFT by reducing resources available

to prefilling and increasing resource contention. This trade-off fundamentally arises between prefill latency and decode throughput under shared system resources, and highlights the need to select appropriate operating points rather than optimizing a single metric.

From the Pareto frontier, we select two representative configurations for further analysis. The configuration labeled B^* is a balanced point that maximizes the throughput to TTFT ratio among configurations whose throughput exceeds the design-space average. The configuration labeled M_T maximizes throughput and represents an aggressively provisioned, throughput-oriented design. As our study targets high-throughput multi-tenant serving systems, subsequent evaluations focus on these two operating points, which reflect the two design objectives considered in this study.

Accuracy of the Markov-based performance estimator: The blue markers in Fig. 8 ($\star$, $\blacktriangle$) indicate the Pareto-optimal configurations predicted by the proposed Markov-based estimator, while the red markers ($\star$, $\blacktriangle$) show the corresponding simulation results. For most workloads, the estimator closely tracks the simulated Pareto frontier, with only modest discrepancies. For example, in the Nemotron-H-Chat workload, the balanced configuration B^* exhibits a 7% throughput gap, while the TTFT of M_T deviates by approximately 17% under static scheduling. These gaps arise because the estimator abstracts away fine-grained queueing effects, transient resource contention, and scheduling decisions, whereas the simulator explicitly models these interactions. Nevertheless, the estimator preserves the relative ordering of configurations and accurately identifies high-quality Pareto candidates, making it effective for pruning inferior designs prior to detailed simulation.

C. Specialization–Generality Trade-Off Study

We further evaluate how well configurations optimized for one hybrid LLM transfer to other hybrid LLMs, exposing the trade-off between model specialization and cross-model generality. This study uses the Chat dataset and three representative hybrid LLM workloads: Jamba-tiny [6] (Jamba), Zamba2-7B [8] (Zamba), and Nemotron-H-4B [7] (Nemo). Jamba contains 2 Attention and 14 Mamba blocks, Zamba contains 13 Attention and 81 Mamba blocks, and Nemotron-H contains 4 Attention and 24 Mamba blocks. For each DSE target, *HYDRA* explores the same macro-architectural design space as in Section VI-B. We consider three specialization levels: (i) single-model DSE, which produces the most specialized configuration, (ii) two-model DSE, which produces a partially generalized configuration, and (iii) three-model DSE, which produces a general configuration optimized across all workloads. The corresponding results are shown as blue, gray, and orange rows in Table IV, respectively.

Table IV reports the resulting specialization–generality trade-off for the two operating objectives in Section VI-B: the balanced point (B^*) and the maximum-throughput point (M_T). Values are normalized to the best result for each workload, so 1.00 denotes the workload-specific optimum. Red entries indicate off-target evaluations, where a configuration is tested on a model excluded from its DSE target.

First, the B^* results in Table IV(a) show that single-model specialization gives the best result on the target workload, but can transfer poorly to other hybrid models. For example, the Jamba-specialized design achieves 1.00 on Jamba, but drops to 0.42 on Zamba and 0.31 on Nemo. This demonstrates that balanced configurations can over-specialize to a particular layer composition and serving behavior. In contrast, mixture-aware DSE improves the weakest transfer cases. The all-model configuration raises Nemo from 0.31 to 0.86 compared with the Jamba-specialized design ($2.77\times$), and raises Zamba from 0.32 to 0.93 compared with the Nemo-specialized design ($2.91\times$). The all-model configuration limits the worst-case degradation to 14% relative to each workload’s specialized optimum.

Second, the M_T results in Table IV(b) show a similar trend, although throughput-oriented designs transfer more effectively because the objective is dominated by sustained decode capacity and batching efficiency. For example, the Zamba-specialized design preserves 97% of Jamba’s maximum throughput and 80% of Nemo’s maximum throughput. However, the Jamba-specialized design remains poorly matched to Zamba and Nemo, reaching only 0.44 and 0.36, respectively, reflecting architectural choices that are tuned to Jamba’s workload characteristics. By comparison, the all-model M_T configuration improves these weak transfer cases to 0.96 and 0.84 ($2.18\times$ and $2.33\times$), while reducing Jamba throughput by only 1%. Similarly, it improves Zamba from 0.62 to 0.96 compared with the Nemo-specialized design, at the cost of reducing Nemo from 1.00 to 0.84.

Overall, these results highlight the deployment tradeoff. For

TABLE IV: Specialization–generality tradeoff across hybrid LLMs. Each row is the DSE target model set, and each column is the evaluation workload. Values are normalized to the workload-specific single-model optimum. (a) reports B^* with normalized TP/TTFT; (b) reports M_T with normalized TP.

(a) B^* : norm. TP/TTFT score $\uparrow$			
DSE Target	Jamba	Zamba	Nemo
Jamba	1.00	0.42	0.31
Zamba	0.91	1.00	0.76
Nemo	0.87	0.32	1.00
Jamba + Zamba	0.97	0.94	0.71
Jamba + Nemo	0.92	0.34	0.93
Zamba + Nemo	0.96	0.93	0.86
Jamba + Zamba + Nemo	0.96	0.93	0.86
(b) M_T : norm. TP $\uparrow$			
DSE Target	Jamba	Zamba	Nemo
Jamba	1.00	0.44	0.36
Zamba	0.97	1.00	0.80
Nemo	0.98	0.62	1.00
Jamba + Zamba	0.97	1.00	0.80
Jamba + Nemo	0.98	0.62	1.00
Zamba + Nemo	0.99	0.96	0.84
Jamba + Zamba + Nemo	0.99	0.97	0.83

Most-specialized Medium-specialized
General off-target evaluation

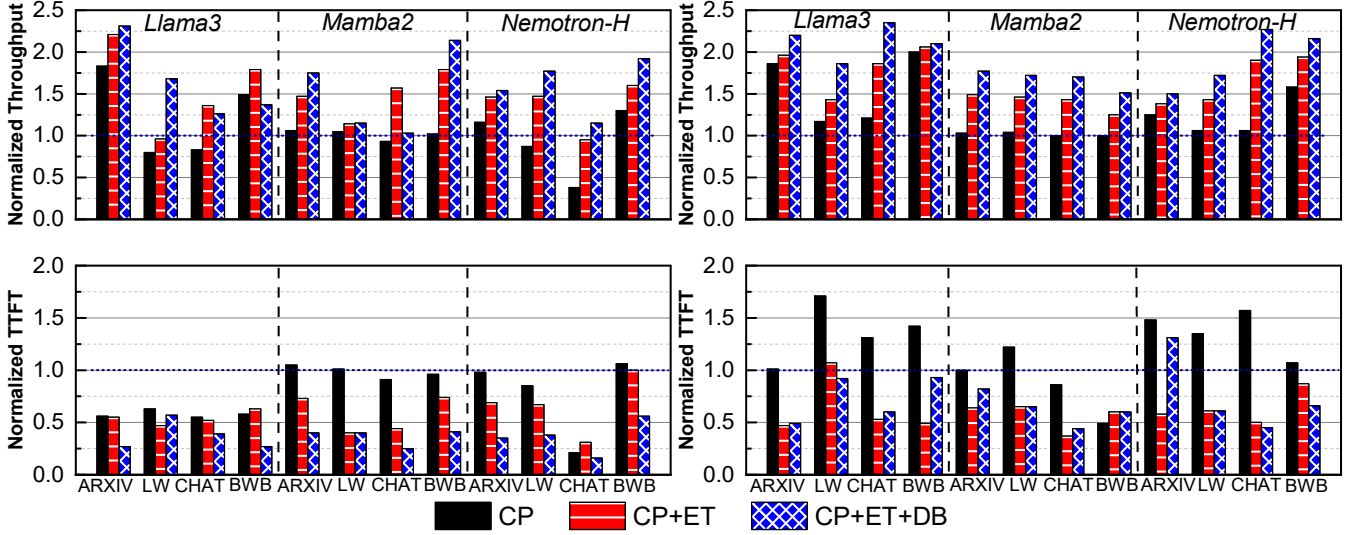


Fig. 9: Ablation study of communication-aware placement (CP, Section IV-A), elastic task scheduling (ET, Section IV-C), and dynamic batching (DB, Section IV-B) strategies. It reports results for 12 model-dataset pairs on the x-axis for the B^* and M_T .

services dedicated to a single stable model, specialized configurations provide the best performance. For platforms expected to support multiple or evolving hybrid LLMs, mixture-aware DSE offers a more robust configuration by substantially improving worst-case performance on unseen models while remaining close to each model’s specialized optimum.

D. Evaluation of Placement Strategies

We next evaluate the impact of chiplet placement strategies on system performance. While the macro-architectural configuration determines available resources, placement governs how efficiently these resources are interconnected. Fig. 9 first reports the effect of communication-aware placement, and the next subsection (Section VI-E) uses Fig. 9 to further evaluate elastic scheduling and dynamic batching for B^* and M_T across all 12 workloads. Results are normalized to a round-robin placement with static scheduling used in Section VI-B.

The black bars correspond to our communication-aware placement (CP) strategy, which clusters chiplets based on their dominant communication patterns. Across all workloads, CP improves throughput by 3%–100% while maintaining comparable or lower TTFT for M_T . For B^* , CP consistently achieves a higher throughput-to-TTFT ratio than the baseline. In several cases, CP simultaneously improves both metrics by enabling larger effective batch sizes. For example, in the Nemotron-H-ArXiv workload under B^* , CP improves throughput by $1.3\times$ with only a $1.06\times$ increase in TTFT, resulting in a net efficiency gain.

To further show the impact of placement, we add a random placement baseline evaluated by the same Nemotron-H workloads and hardware configurations as in Section VI-B. All placement baselines keep memory chiplets on the outer ring of the interposer. The round-robin (RR) baseline distributes accelerator chiplets and model weights in round-robin order, while the random baseline randomly places accelerator chiplets and randomly assigns model weights to HBM chiplets.

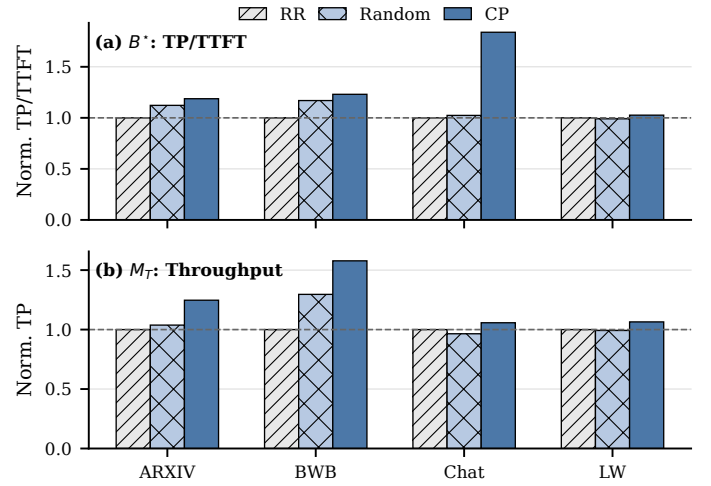


Fig. 10: Comparison of chiplet placement strategies across Nemotron-H workloads.

Fig. 10 reports the resulting TP/TTFT for B^* and throughput for M_T on the ARXIV, BWB, Chat, and LongWriter datasets. Random placement behaves similarly to RR, indicating that simply distributing chiplets across the interposer does not consistently improve communication locality. On average, CP improves TP/TTFT by $1.29\times$ and throughput by $1.22\times$ relative to RR. Compared with random placement, CP improves TP/TTFT by $1.04\times$ – $1.80\times$ and throughput by $1.07\times$ – $1.22\times$. These results show that robust placement gains require explicitly aligning compute chiplets with their dominant HBM communication partners rather than relying on arbitrary chiplet distributions.

To understand the source of these improvements, Fig. 11 visualizes NoI link-utilization heatmaps for the Nemotron-H-LW workload under two placement strategies. The round-robin placement (left) exhibits widespread D2D bandwidth pressure, indicating frequent long-distance communication be-

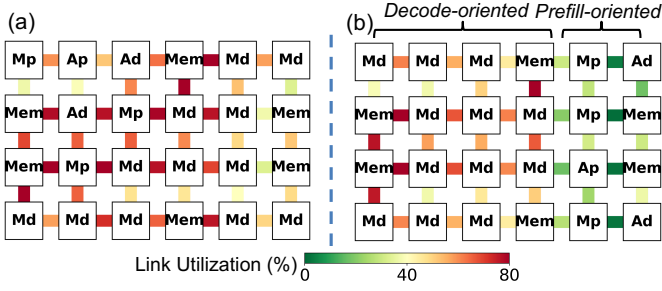


Fig. 11: NoI heatmaps of bandwidth utilization that compare the round-robin (a) and communication-aware placement (b).

tween HBM and compute chiplets. In contrast, CP (right) significantly reduces bandwidth pressure, as reflected by lighter-colored links, by minimizing average hop counts and localizing data movement. Although CP reduces average hop count and removes widespread NoI pressure, it cannot eliminate hotspots created by the intrinsic bandwidth demand of steady-state decoding. The remaining high-utilization links are therefore expected under fixed NoI bandwidth. In addition, CP spatially separates prefilling-oriented and decoding-oriented chiplets, which reduces cross-phase interference on shared NoI resources. While this separation can slightly increase TTFT in some cases due to reduced opportunistic sharing, it substantially improves throughput by mitigating sustained NoI congestion during steady-state decoding.

This observation demonstrates that reducing communication distance is more effective than simply increasing compute parallelism, highlighting that communication locality, rather than raw compute provisioning, is often the dominant bottleneck in chiplet-based LLM serving. This aligns with *HYDRA*'s design philosophy of co-optimizing architecture and data movement.

Together, these results show that chiplet placement plays a critical role in establishing baseline system behavior for a given macro-architectural configuration. However, placement alone cannot adapt to dynamic request arrivals or transient resource imbalance. The next subsection shows how elastic task scheduling and dynamic batching further build on these gains under dynamic workloads.

E. Evaluation of Elastic Scheduling and Dynamic Batching

Fig. 9 also evaluates the impact of runtime elastic task scheduling and dynamic batching on the two representative configurations, B^* and M_T , across all 12 workloads. The red horizontally hatched bars enable runtime elastic task scheduling on top of static batching, while the blue crosshatched bars further incorporate dynamic request batching.

Elastic scheduling improves performance across all workloads by dynamically redistributing work across chiplets when runtime imbalance arises. Under static mapping, task assignments are fixed offline, and transient effects, such as prolonged decoding phases or bursty request arrivals, can create pipeline stalls and leave resources underutilized. Elastic scheduling mitigates this by migrating ready tasks to underutilized chiplets, guided by task-accelerator affinity and bounded communication

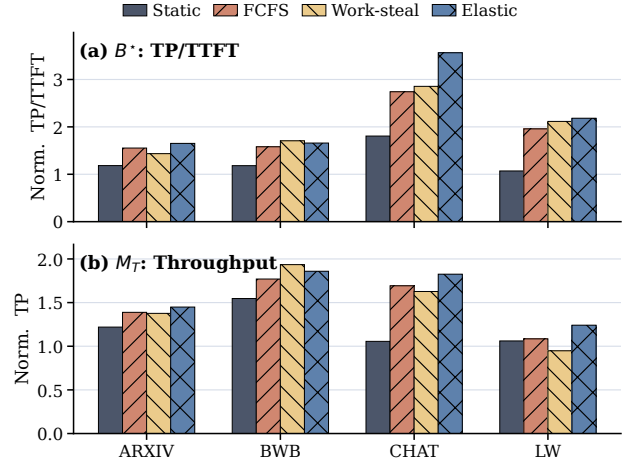


Fig. 12: Comparison of task schedulers across Nemotron-H workloads.

overhead. This enables adaptive load balancing, leading to 3%–150% higher throughput and $1.02\times$ – $3.14\times$ lower TTFT.

To further evaluate the effect of task scheduling, we compare four task-mapping policies using the same Nemotron-H workloads and hardware configurations as in Section VI-B. For each workload, only the runtime task scheduler is changed. Static mapping always follows the placement-selected chiplet assignment. The First-Come-First-Served (FCFS) baseline [48] assigns each incoming task to a compatible chiplet in round-robin order, ignoring placement preference and locality. Work-stealing [49] dynamically migrates queued tasks from heavily loaded chiplets to lightly loaded compatible chiplets to improve load balance. In contrast, *HYDRA*'s elastic scheduler preserves the placement-preferred chiplet when the imbalance is mild, but selectively reassigns tasks based on both queue occupancy and communication distance to the associated memory chiplets. This design retains communication locality while still exploiting dynamic load balancing opportunities.

Fig. 12 reports the resulting TP/TTFT for B^* and throughput for M_T across the ARXIV, BWB, Chat, and LongWriter datasets. Compared with static mapping, elastic scheduling improves TP/TTFT by $1.40\times$ – $2.04\times$ and improves maximum throughput by $1.17\times$ – $1.73\times$. Compared with FCFS, elastic scheduling improves the target metric by $1.04\times$ – $1.30\times$ in all cases. Compared with work-stealing, elastic scheduling achieves the best result in six of the eight cases, improving the target metric by up to $1.31\times$. In the two remaining BWB cases, elastic scheduling remains within 4% of work-stealing. This is because work-stealing aggressively reacts to queue imbalance, but it is not communication-aware. Overall, these results show that elastic scheduling provides most of the load-balancing benefit of aggressive dynamic reassignment while considering communication locality, unlike FCFS and work-stealing.

Dynamic batching further amplifies these gains by increasing the effective batch sizes at runtime. By decoupling memory allocation from admission through virtualized memory blocks, *HYDRA* can admit new requests as long as aggregate compute and memory budgets permit. The batching controller contin-

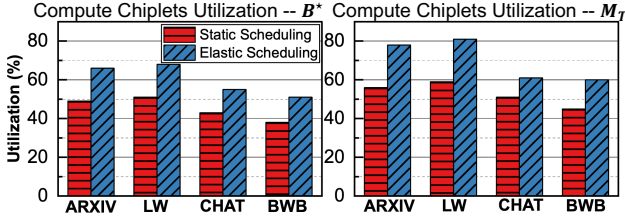


Fig. 13: Comparison of compute-chiplet utilization across Nemotron-H workloads.

uously monitors token-level progress and adjusts the number of in-flight requests to maximize utilization while bounding TTFT increase. Dynamic batching improves memory utilization and throughput of attention-based models, whose KV-cache footprint grows with sequence length. Though SSM-based models exhibit relatively fixed memory footprints determined by state dimensions, they still benefit from continuous request batching. Across all workloads, dynamic batching achieves a 15% throughput gain while reducing TTFT by 23% on average. **Quantitative summary:** Elastic scheduling primarily improves load balance, while dynamic batching improves timely resource allocation. This combination improves throughput by up to $2.3\times$ relative to the static baseline, while reducing TTFT to as low as 40%. These improvements correlate with higher and more balanced compute-chiplet utilization, as shown in Fig. 13 for Nemotron-H across four datasets. These results demonstrate that runtime adaptivity is essential for fully exploiting optimized macro-architectural configurations. While placement establishes efficient communication structure, dynamic scheduling and batching are required to handle workload variability and achieve sustained high utilization in multi-tenant LLM serving.

F. Performance Model for Fast DSE

Table V compares the efficiency and effectiveness of different design-space exploration strategies. Exhaustive simulation evaluates all feasible configurations under dynamic arrivals, batching, and scheduling, but incurs prohibitively long runtimes, ranging from *multiple days to over a week*, due to detailed NoI communication, memory management, and runtime contention models. In contrast, the roofline-based approach completes within minutes by relying on static kernel bounds and idealized execution assumptions, but its limited modeling fidelity leads to suboptimal design choices. The best configuration it finds achieves only 74% of the achievable throughput.

The proposed *HYDRA* framework, powered by the fast Markov-based performance estimator, achieves both high efficiency and high accuracy. By explicitly incorporating workload characteristics and runtime behavior, such as elastic task scheduling and dynamic batching, into a lightweight analytical estimator, *HYDRA* identifies high-quality configurations within 4–15 minutes exploration time comparable to the roofline baseline.

HYDRA recovers nearly the full performance potential of exhaustive simulation, identifying a configuration whose throughput is 96% of the best configuration found by exhaustive simulation. Moreover, the Pareto-optimal set closely tracks the

exhaustive Pareto frontier and contains optimal configurations in some cases. These results demonstrate that the Markov-based estimator accurately captures the dominant runtime dynamics that govern multi-tenant LLM serving performance. This enables efficient search while supporting fast, accurate, and scalable exploration of system-level trade-offs in heterogeneous chiplet architectures for LLM serving.

TABLE V: Average exploration time for different search strategies and maximum-throughput recovery. Each exploration covers four datasets per model.

Model	Exhaustive	Roofline [32]	HYDRA
LLAMA3-7B	2 d. 8 h.	4 min	4 min
MAMBA2-2.8B	3 d. 1 h.	4 min	4 min
Nemotron-H-4B	8 d. 8 h.	15 min	15 min
Max Throughput	100%	74%	96%

VII. CONCLUSION

This paper presented *HYDRA*, a design space exploration framework for heterogeneous chiplet-based systems targeting dynamic hybrid LLM serving. *HYDRA* jointly models chiplet placement, request batching, runtime scheduling, and fast performance estimation to capture the interaction between architecture and workload dynamics. Across hybrid, Mamba, and Transformer workloads, the proposed optimization strategies improve throughput by up to $2.3\times$ while lowering TTFT by 40%, and the Markov-based estimator enables efficient exploration that reduces DSE time from days to minutes. These results show that effective hybrid LLM serving requires co-optimizing compute, memory, communication, and runtime control policies. Overall, *HYDRA* provides a practical and scalable framework for exploring heterogeneous chiplet architectures for next-generation multi-tenant LLM systems.

This work focuses on throughput and TTFT as the primary serving metrics. Power, thermal, and reliability are also important design considerations, particularly as chiplet counts and communication bandwidth continue to increase. Since *HYDRA* already models chiplet placement, communication activity, memory utilization, and runtime scheduling, the framework can be naturally extended with power and thermal models to support power-aware and thermal-aware design space exploration in future work.

REFERENCES

- [1] A. Gu, K. Goel, and C. Re, “Efficiently modeling long sequences with structured state spaces,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.
- [2] A. Gu and T. Dao, “Mamba: Linear-time sequence modeling with selective state spaces,” in *Proceedings of the Conference on Language Modeling (COLM)*, 2024.
- [3] J. Li *et al.*, “MARCA: Mamba accelerator with reconfigurable architecture,” in *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2024, pp. 1–9.
- [4] R. Wei *et al.*, “LightMamba: Efficient Mamba acceleration on FPGA with quantization and hardware co-design,” in *Proceedings of the Design, Automation and Test in Europe Conference (DATE)*, 2025, pp. 1–7.
- [5] J. Kim *et al.*, “eMamba: Efficient acceleration framework for Mamba models in edge computing,” *ACM Transactions on Embedded Computing Systems*, vol. 24, no. 5s, pp. 1–22, 2025.

- [6] B. Lenz *et al.*, “Jamba: Hybrid transformer-mamba language models,” in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 67 959–67 984.
- [7] A. Blakeman *et al.*, “Nemotron-h: A family of accurate and efficient hybrid mamba-transformer models,” *arXiv preprint arXiv:2504.03624*, 2025.
- [8] P. Gloriosio *et al.*, “Zamba: A compact 7b ssm hybrid model,” *arXiv preprint arXiv:2405.16712*, 2024.
- [9] A. Amirshahi, G. Ansaloni, and D. Atienza, “Accelerator-driven data arrangement to minimize transformers run-time on multi-core architectures,” in *Proceedings of the 15th Workshop on Parallel Programming and Run-Time Management Techniques for Many-Core Architectures and the 13th Workshop on Design Tools and Architectures for Multicore Embedded Computing Platforms*, 2024.
- [10] Y. Zhong *et al.*, “DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving,” in *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2024)*, 2024, pp. 193–210.
- [11] H. Sharma *et al.*, “A heterogeneous chiplet architecture for accelerating end-to-end transformer models,” *ACM Transactions on Design Automation of Electronic Systems*, vol. 30, no. 3, pp. 1–24, 2025.
- [12] Z. Xu *et al.*, “WSC-LLM: Efficient LLM service and architecture co-exploration for wafer-scale chips,” in *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA)*, 2025, pp. 1–17.
- [13] Advanced Micro Devices, Inc., “AMD Ryzen™ Threadripper™ PRO Processors for Architecture, Engineering, and Construction (AEC),” accessed 4/21/2026. [Online]. Available: <https://www.amd.com/content/dam/amd/en/documents/products/processors/ryzen-threadripper/threadripper-pro-design-aec.pdf>
- [14] B. Hu *et al.*, “Blockllm: Multi-tenant finer-grained serving for large language models,” *arXiv preprint arXiv:2404.18322*, 2024.
- [15] W. Kwon *et al.*, “Efficient memory management for large language model serving with PagedAttention,” in *Proc. of 29th Symposium on Operating Systems Principles (SOSP)*, 2023, pp. 611–626.
- [16] G.-I. Yu *et al.*, “Orca: A distributed serving system for transformer-based generative models,” in *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2022)*, 2022, pp. 521–538.
- [17] A. Agrawal *et al.*, “Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve,” in *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2024)*, 2024, pp. 117–134.
- [18] J. Cai *et al.*, “Gemini: Mapping and architecture co-exploration for large-scale DNN chiplet accelerators,” in *Proceedings of the IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2024, pp. 156–171.
- [19] M. J. Adiletta, G.-Y. Wei, and D. Brooks, “Democratizing customization for ML at the edge through hetero-chiplet SiP architectures,” *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, 2025.
- [20] A. Taghibakhshi *et al.*, “Efficient hybrid language model compression through group-aware ssm pruning,” *Advances in Neural Information Processing Systems*, vol. 38, pp. 122 956–122 976, 2026.
- [21] J. Liu *et al.*, “TSTC: Two-level sparsity tensor core enabling both algorithm flexibility and hardware efficiency,” in *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2023, pp. 1–9.
- [22] S. Tuli and N. K. Jha, “AccelTran: A sparsity-aware accelerator for dynamic inference with transformers,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 42, no. 11, pp. 4038–4051, 2023.
- [23] S. Ko and K. Olukotun, “Ssm-rdu: A reconfigurable dataflow unit for long-sequence state-space models,” in *2025 IEEE 43rd International Conference on Computer Design (ICCD)*. IEEE, 2025, pp. 626–629.
- [24] L. Zhong *et al.*, “Specmamba: Accelerating mamba inference on fpga with speculative decoding,” in *2025 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. IEEE, 2025, pp. 1–9.
- [25] R. Geens, A. Symons, and M. Verhelst, “Fine-grained fusion: The missing piece in area-efficient state space model acceleration,” in *2025 34th International Conference on Parallel Architectures and Compilation Techniques (PACT)*. IEEE, 2025, pp. 281–291.
- [26] A. Agrawal *et al.*, “Vidur: A large-scale simulation framework for LLM inference,” *Proceedings of Machine Learning and Systems*, vol. 6, pp. 351–366, 2024.
- [27] NVIDIA, “NVIDIA blackwell architecture,” <https://resources.nvidia.com/en-us-blackwell-architecture/datasheet>, 2024.
- [28] M.-J. Park *et al.*, “A 192-gb 12-high 896-gb/s HBM3 DRAM with a TSV auto-calibration scheme and machine-learning-based layout optimization,” *IEEE Journal of Solid-State Circuits*, vol. 58, no. 1, pp. 256–269, 2023.
- [29] “JESD239A: Graphics double data rate (GDDR7) SGRAM devices,” <https://store.accuris.tech.com/standards/jedec-jesd239a>, JEDEC Solid State Technology Association, Tech. Rep., 2024.
- [30] A. Kannan, N. E. Jerger, and G. H. Loh, “Enabling interposer-based disintegration of multi-core processors,” in *Proceedings of the 48th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2015, pp. 546–558.
- [31] AMD, “AMD Instinct MI350 Series GPUs: A game changer for inference, training and HPC workloads,” <https://www.amd.com/en/blogs/2025/amd-instinct-mi350-series-game-changer.html>, Sep. 2025.
- [32] S. Williams, A. Waterman, and D. Patterson, “Roofline: an insightful visual performance model for multicore architectures,” *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.
- [33] G. Bolch *et al.*, *Queueing Networks and Markov Chains: Modeling and Performance Evaluation with Computer Science Applications*. John Wiley and Sons, 2006.
- [34] N. Bauerle, “Optimal control of queueing networks: An approach via fluid models,” *Advances in Applied Probability*, vol. 34, no. 2, pp. 313–328, 2002.
- [35] A. Grattafiori *et al.*, “The llama 3 herd of models,” *arXiv preprint arXiv:2407.21783*, 2024.
- [36] A. Cohan *et al.*, “A discourse-aware attention model for abstractive summarization of long documents,” in *Proceedings of NAACL-HLT 2018, Volume 2 (Short Papers)*, 2018, pp. 615–621.
- [37] Y. E. Jiang *et al.*, “Discourse-centric evaluation of document-level machine translation with a new densely annotated parallel corpus of novels,” in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2023.
- [38] Y. Bai *et al.*, “LongWriter: Unleashing 10,000+ word generation from long context LLMs,” in *Proceedings of the Thirtieth International Conference on Learning Representations (ICLR)*, 2025.
- [39] L. Zheng *et al.*, “Lmsys-chat-1m: A large-scale real-world llm conversation dataset,” in *International Conference on Learning Representations*, vol. 2024, 2024, pp. 22 225–22 257.
- [40] Y. S. Shao *et al.*, “Simba: Scaling deep-learning inference with multi-chip-module-based architecture,” in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2019, pp. 14–27.
- [41] J. W. Poulton *et al.*, “A 1.17-pj/b, 25-gb/s/pin ground-referenced single-ended serial link for off- and on-package communication using a process- and temperature-adaptive voltage regulator,” *IEEE Journal of Solid-State Circuits*, vol. 54, no. 1, pp. 43–54, 2019.
- [42] D. D. Sharma *et al.*, “Universal chiplet interconnect express (UCIe): An open industry standard for innovations with chiplets at package level,” *IEEE Transactions on Components, Packaging and Manufacturing Technology*, vol. 12, no. 9, pp. 1423–1431, 2022.
- [43] Y. Feng and K. Ma, “Chiplet actuary: A quantitative cost model and multi-chiplet architecture exploration,” in *Proceedings of the 59th ACM/IEEE Design Automation Conference (DAC)*, 2022, pp. 121–126.
- [44] A. Stillmaker and B. Baas, “Scaling equations for the accurate prediction of CMOS device performance from 180 nm to 7 nm,” *Integration, the VLSI Journal*, vol. 58, pp. 74–81, 2017.
- [45] H. Luo *et al.*, “Ramulator 2.0: A modern, modular, and extensible DRAM simulator,” *IEEE Computer Architecture Letters*, vol. 23, no. 1, pp. 112–116, 2024.
- [46] R. Balasubramonian *et al.*, “CACTI 7: New tools for interconnect exploration in innovative off-chip memories,” *ACM Transactions on Architecture and Code Optimization*, vol. 14, no. 2, pp. 1–25, 2017.
- [47] R. Andri, E. Reggiani, and L. Cavigelli, “Flex-SFU: Activation function acceleration with non-uniform piecewise approximation,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2025.
- [48] A. Silberschatz, *et al.*, *Operating System Concepts*, 10th ed. Wiley, 2018.
- [49] T. Chen *et al.*, “An architectural framework for accelerating dynamic parallel algorithms on reconfigurable hardware,” in *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2018, pp. 55–67.

APPENDIX

The HYDRA artifact provides the event-driven simulator, analytical accelerator models, hybrid-LLM workload traces, hardware configurations, fast-DSE models, curated measurements, and scripts used in the evaluation. It supports the standard experiments associated with Sections VI-B–VI-F, including macro-architecture DSE, the specialization–generality study, placement and scheduling ablations, dynamic batching, and fast DSE. The standard evaluation runs on a CPU-only Linux machine, regenerates result tables and PNG figures, and reports the status and output location of each experiment. Expensive exhaustive DSE sweeps are provided as optional, resumable workflows.

A. Artifact Checklist

Program: HYDRA event-driven simulator, analytical models, experiment runners, and visualization scripts.

Compilation: None; the standard workflow is Python-based.

Models: Llama3-7B, Mamba2-2.8B, Nemotron-H-4B, Jamba-tiny and Zamba2-7B hybrid-LLM configurations.

Data sets: Preprocessed request-length traces from ARXIV, BWB, Chat, and LongWriter; no raw prompt or generated text is included.

Run-time environment: Linux x86-64, Python 3.11, Bash, and the pinned packages in `requirements.txt`.

Hardware: CPU-only; 8 CPU cores, 16 GB memory, and 10 GB free disk space are recommended for the standard workflow.

Execution: Automated Bash wrappers invoking Python simulation and post-processing scripts.

Metrics: Throughput, time to first token (TTFT), throughput/TTFT, chiplet utilization, and fast-DSE recovery.

Output: CSV result tables, PNG figures, simulator outputs, and per-experiment logs.

Experiments: Figures 8–10 and 12, Table IV, and the fast-DSE evaluation in Sections VI-B–VI-F.

Time: Approximately 10–20 minutes for environment setup and 7 minutes for the functional workflow using completed outputs.

Publicly available: Yes; the repository link is given below.

License: The software license is provided in the repository’s `LICENSE` file.

Workflow automation: Repository-provided Bash and Python scripts; no external workflow framework is required.

B. Description

1) *How to Access:* The artifact is publicly available at <https://github.com/ONQLin/hydra-cases-2026-artifact>. Clone this repository and enter its root directory before following the instructions in `INSTALL`.

The top-level `README.md` introduces the repository, while `REQUIREMENTS`, `INSTALL`, and `STATUS` define the supported environment, installation test, and evaluation scope. A copy of the accepted manuscript is included as `HYDRA_paper.pdf`.

C. Installation and Workflow

Detailed installation and per-experiment instructions are provided in `INSTALL` and `artifact/README.md`. The minimal setup is:

```
python3.11 -m venv .venv
```

```
source .venv/bin/activate
python -m pip install -r requirements.txt
```

Run the standard functional-evaluation workflow with:

```
HYDRA_REUSE_COMPLETED=1 \
bash artifact/reproduce_all.sh
```

This command runs all non-optional experiments in paper order. Individual experiment entry points and the optional exhaustive workflows are listed in `artifact/README.md`. The `HYDRA_NUM_WORKERS` environment variable controls simulation concurrency.

D. Evaluation and Expected Results

During execution, `artifact/reproduce_all.sh` prints the active experiment, output directories, elapsed time, and PASS/FAIL status. A successful evaluation ends with:

```
HYDRA reproduction completed: PASS
Figures: artifact/figures/
Data:    artifact/run_outputs/
```

Generated CSV files and PNG figures are written under `artifact/run_outputs/` and `artifact/figures/`, respectively. Timestamped logs and a tabular summary are written under `output_temp_runs/artifact_reproduction/`. Small machine-dependent numerical differences do not change the reported comparisons or trends.